SISTEMA NATYS

Informe de Pruebas de Software

Módulo: Carrito de compra

Integrante	Elias Arma Moises Pereira Edgmairys Aponte Ismael Torres Santiago Loyo
Equipo	Equipo 8
Docente	Profa. Angelismar Terán
Sección	IN3103
Fecha	17-10-2026

1. Introducción

El presente informe documenta el proceso integral de aseguramiento de calidad (QA) y pruebas de software aplicadas a cada uno de los módulos que integran el **Sistema Naty's**. El objetivo principal es verificar y validar el correcto funcionamiento de los flujos de trabajo del sistema, incluyendo la gestión completa del ciclo de vida de los datos (registro, consulta, modificación, eliminación, aprobación y cancelación de acciones).

Para garantizar un nivel óptimo de estabilidad, rendimiento y seguridad, se implementó una estrategia de evaluación multinivel que abarca las siguientes tipologías de pruebas:

- **Pruebas Unitarias y de Humo:** Ejecutadas directamente sobre la capa de modelo mediante **PHPUnit** para verificar el comportamiento aislado de las funciones core y asegurar la estabilidad básica antes de fases avanzadas.
- **Pruebas de Integración y Aceptación:** Realizadas con **PHPUnit** y **Codeception**, validando la interacción entre componentes y garantizando que los módulos cumplan con los criterios de aceptación y requerimientos del usuario.
- **Pruebas de Backend y Seguridad:** Implementadas con **Postman** para auditar el comportamiento de las APIs, el manejo de respuestas HTTP y la resistencia ante vulnerabilidades en los endpoints.
- **Pruebas de Interfaz de Usuario (UI):** Automatizadas con **Selenium** para simular la navegación e interacción directa del usuario en el navegador.
- **Pruebas de Rendimiento y Carga:** Llevadas a cabo mediante **Apache JMeter**, evaluando los tiempos de respuesta y la estabilidad del sistema bajo volúmenes de concurrencia y carga de datos.

Modulo: Carrito de compra

La primera prueba (UT-01) detectó un defecto en el registro de entregas, que se corrigió y se verificó con una re-ejecución. Luego se completó la cobertura del CRUD con cuatro pruebas más (UT-02 a UT-05). Todas las pruebas unitarias se aislaron con mocks, de modo que se ejecutan sin servidor ni base de datos. Finalmente, una prueba de integración (IT-01) verificó el modelo contra la base de datos real y detectó un segundo defecto (BUG-02), también corregido.

1.1 Entorno de pruebas

Sistema operativo	Windows 11 (compilación 26200)
Servidor local	XAMPP con MySQL activo en la ejecución inicial de UT-01; apagado en todas las ejecuciones aisladas
Lenguaje	PHP 8.2.12
Herramienta	PHPUnit 11.5.57 (instalado con Composer), con mocks de PHPUnit
Configuración	phpunit.xml · bootstrap: vendor/autoload.php · carpeta de pruebas: tests

Proyecto	C:\xampp\htdocs\natys-larence

1.2 Resumen de resultados

ID	Caso de prueba	Acción	Estado
ID	**Caso de prueba**	**Acción**	**Estado**
UT-01	Acción inválida retorna error de estructura	manejarAccion	**Aprobado**
UT-02	Obtener carrito activo crea o retorna carrito mediante mock	obtener_activo	**Aprobado**
UT-03	Registrar producto en carrito de compra	agregar	**Aprobado**
UT-04	Modificar cantidad o ítems en el carrito de compra	modificar	**Aprobado**
UT-05	Eliminar producto específico del carrito de compra	eliminar	**Aprobado**
Suite	Archivo completo tests/unitarias/CarritoTest.php (5 pruebas unitarias)	todas	**5 de 5 Aprobado**
IT-01	Integración: modificar producto y persistencia en la base real	modificar	**Aprobado**
API-01	Backend: consultar el carrito activo y sus ítems (Postman)	consultar	**Aprobado**
SM-01	Humo: el módulo de Carrito inicializa y carga correctamente	todas	**Aprobado**
UI-01	Interfaz y sistema: Modificar cantidades actualiza la vista (Selenium IDE)	modificar	**Aprobado**
SEC-01	Seguridad: consultar el carrito sin sesión activa (Postman)	consultar	**Aprobado**
SEC-02	Seguridad: inyección SQL en parámetros del carrito (Postman)	obtener	**Aprobado**
RD-01	Rendimiento: 50 usuarios consultan el carrito simultáneamente (JMeter)	login / consultar	**Aprobado**
AC-01	Aceptación: acceder al carrito y verificar el selector de método de pago (Codeception)	consultar	**Aprobado**

1.3 Cobertura del CRUD

Operación	Prueba que la cubre	Comportamiento verificado
Crear / Registrar	UT-03 · Registrar producto en carrito	**Valida stock, cajas y datos obligatorios antes de insertar en el detalle del carrito.**
Leer (una / activo)	UT-02 · Obtener carrito activo mediante mock	**Crea o retorna el carrito activo del usuario consultando con éxito mediante PDO.**
Leer (listado)	UT-01 · Acción inválida / Consulta del carrito	**Retorna los ítems, subtotales y la estructura correcta del carrito.**
Actualizar	UT-04 · Modificar producto en el carrito	**Valida el stock disponible, actualiza la cantidad y confirma**

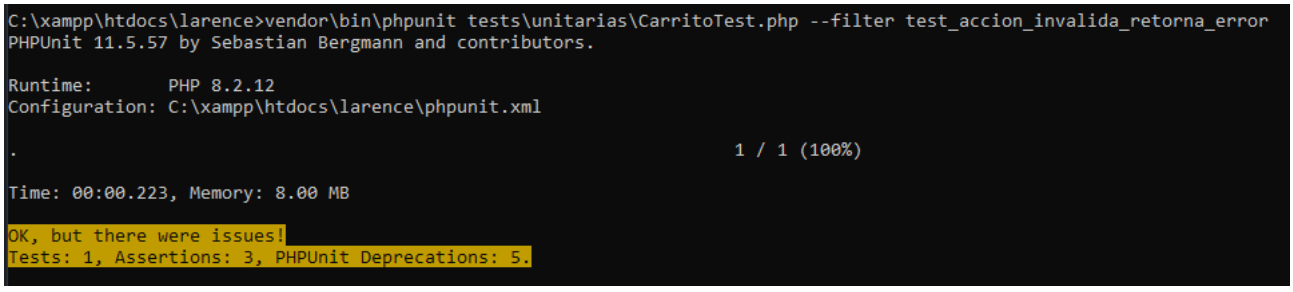
Operación	Prueba que la cubre	Comportamiento verificado
		la transacción de forma segura.
Eliminar	UT-05 · Eliminar producto en el carrito	Remueve el producto seleccionado del detalle del carrito y confirma la transacción.

2. Prueba unitaria UT-01

Campo	Detalle
ID	UT-01
Nombre	Acción inválida retorna error de estructura
Tipo de prueba	Unitaria
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/unitarias/CarritoTest.php · test_accion_invalida_retorna_error
Descripción	Verifica que el método centralizador manejarAccion rechace cualquier comando o acción no reconocida ('accion_falsa'), devolviendo un arreglo con status = false y un mensaje de control sin afectar la base de datos.
Precondiciones	PHPUnit instalado en el proyecto · Configuración phpunit.xml presente · Entorno aislado sin requerir servidor MySQL activo.
Datos de entrada	**Acción:** accion_falsa **JSON:** {}
Pasos	1. Se abrió la terminal CMD en la ruta del proyecto C:\xampp\htdocs\larence. 2. Se verificó PHPUnit con: vendor\bin\phpunit --version 3. Se ejecutó la prueba unitaria específica: vendor\bin\phpunit tests\unitarias\CarritoTest.php --filter test_accion_invalida_retorna_error
Resultado esperado	El modelo procesa la acción desconocida, devuelve un arreglo con status = false y el mensaje de error 'Accion Invalida.'.
Resultado obtenido	El modelo devolvió correctamente status = false y el mensaje esperado. PHPUnit validó satisfactoriamente todas las aserciones de la estructura de error.
Estado	**Aprobado**
Fecha de ejecución	16-10-2026
Ejecutado por	Moisés Pereira

2.1 Evidencias

Captura 1 – Ejecución de UT-01



Captura 2 – Método de prueba aislado

```php
/** @test */
public function test_accion_invalida_retorna_error(): void {
    $carrito = new Carrito();
    $resultado = $carrito->manejarAccion('accion_falsa', '{}');

    $this->assertIsArray($resultado);
    $this->assertFalse($resultado['status']);
    $this->assertEquals('Accion Invalida.', $resultado['msj']);
}
```

test_accion_invalido_retorna_error en tests/CarritoTest.php

2.2. Prueba unitaria UT-02: Obtener carrito activo crea o retorna carrito mediante mock

ID	UT-02
Nombre	Obtener carrito activo crea o retorna carrito mediante mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	obtener_activo
Archivo y método	tests/unitarias/CarritoTest.php · test_obtener_activo_crea_o_retorna_carrito_con_mock
Descripción	Verifica que el método obtener_activo busque un carrito existente en estado activo para el usuario; en caso de no existir, simula una inserción controlada mediante PDO y retorna los datos del carrito recién creado con status = true.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · PHPUnit configurado y operativo en el proyecto.
Simulación (mock)	Se simuló el comportamiento de la conexión PDO y PDOStatement para interceptar la consulta de búsqueda (SELECT) devolviendo false (inexistente), forzando el INSERT controlado y capturando el ID simulado mediante lastInsertId.
Datos de entrada	**Acción:** `obtener_activo` **JSON:** `{"id_usuario": 1, "id_cliente": 5}`
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/CarritoTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento total. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\CarritoTest.php --filter test_obtener_activo_crea_o_retorna_carrito_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo devuelve status = true, el mensaje «Carrito obtenido.» y los datos correspondientes al carrito activo (id_carrito y id_usuario) sin requerir conexión física a la base de datos.
Resultado obtenido	✔️ Obtener activo crea o retorna carrito con mock. 1 prueba, varias aserciones correctas, sin fallos. El modelo devolvió el estado y los datos esperados mediante el aislamiento con mocks.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Codigo del método aislado

```php
public function test_obtener_activo_crea_o_retorna_carrito_con_mock(): void {
    // 1. Mock de PDOStatement para SELECT del carrito activo
    $stmtSelectMock = $this->createMock(PDOStatement::class);
    $stmtSelectMock->expects($this->once())->method('bindValue');
    $stmtSelectMock->expects($this->once())->method('execute')->willReturn(true);
    $stmtSelectMock->expects($this->once())->method('fetch')->willReturn(false); // Forzar creación (INSERT)

    // 2. Mock de PDOStatement para INSERT de nuevo carrito
    $stmtInsertMock = $this->createMock(PDOStatement::class);
    $stmtInsertMock->expects($this->atLeastOnce())->method('bindValue');
    $stmtInsertMock->expects($this->once())->method('execute')->willReturn(true);

    // 3. Mock de PDO con callback inteligente para prepare
    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtSelectMock, $stmtInsertMock) {
        if (strpos($sql, 'SELECT id_carrito') !== false) {
            return $stmtSelectMock;
        }
        if (strpos($sql, 'INSERT INTO carrito') !== false) {
            return $stmtInsertMock;
        }
        return $this->createMock(PDOStatement::class);
    });

    $pdoMock->expects($this->once())->method('lastInsertId')->willReturn('10');

    // 4. Mock parcial de la clase Carrito
    $carritoMock = $this->getMockBuilder(Carrito::class)
                        ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                        ->getMock();

    $carritoMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $carritoMock->method('closeConnection')->willReturn(null);

    // 5. Ejecutar acción 'obtener_activo'
    $jsonDatos = json_encode([ 'id_usuario' => 1, 'id_cliente' => 5 ]);
    $resultado = $carritoMock->manejarAccion('obtener_activo', $jsonDatos);

    // 6. Aserciones
    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
    $this->assertEquals('Carrito obtenido.', $resultado['msj']);
    $this->assertEquals(10, $resultado['data']['id_carrito']);
    $this->assertEquals(1, $resultado['data']['id_usuario']);
}
```

Captura 6 – Ejecución de UT-02

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\CarritoTest.php --filter test_obtener_activo_crea_o_retorna_carrito_con_mock
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.058, Memory: 10.00 MB

OK, but there were issues!
Tests: 1, Assertions: 8, PHPUnit Deprecations: 5.
```

Prueba aprobada con XAMPP apagado

2.3 Prueba unitaria UT-03: Registrar producto en el carrito de compra mediante

ID	UT-03
Nombre	Registrar producto en el carrito de compra
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	agregar
Archivo y método	tests/unitarias/CarritoTest.php · test_agregar_producto_exitoso
Descripción	Verifica que el método agregar valide el stock disponible del producto, compruebe los topes máximos de cajas permitidas y registre correctamente el ítem en el detalle del carrito activo (insertando un nuevo registro o actualizando su cantidad), confirmando la transacción sin requerir base de datos física.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · CarritoTest.php configurado con mocks y callback de PDO.
Simulación (mock)	Se simuló el comportamiento de la conexión PDO para interceptar la búsqueda del carrito activo, la obtención de la información y stock del producto, el cálculo de las cajas actuales y la ejecución correcta del INSERT sobre la tabla detalle_carrito.
Datos de entrada	**`Acción: agregar`** **`JSON: {"id_usuario": 1, "producto": 3, "cantidad": 2}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/CarritoTest.php. 2. Se detuvieron Apache y MySQL en el panel de control de XAMPP para aislar el entorno. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\CarritoTest.php --filter test_agregar_producto_exitoso --testdox 4. Se registró la salida obtenida en la consola.
Resultado esperado	El modelo procesa la solicitud, valida que el producto exista y tenga stock suficiente, devuelve status = true con un mensaje indicando que el producto fue agregado al carrito y ejecuta con éxito la transacción.
Resultado obtenido	✔️ Agregar producto exitoso. 1 prueba ejecutada con múltiples aserciones correctas, sin fallos. El modelo registró el producto y confirmó la transacción simulada de forma óptima.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – COdigo del método aislado

```php
/** @test */
public function test_agregar_producto_exitoso(): void {
    // Stmts simulados para cada consulta interna de Agregar()
    $stmtCarrito = $this->createMock(PDOStatement::class);
    $stmtCarrito->method('execute')->willReturn(true);
    $stmtCarrito->method('fetch')->willReturn(['id_carrito' => 10, 'id_usuario' => 1, 'id_cliente' => 5]);

    $stmtProd = $this->createMock(PDOStatement::class);
    $stmtProd->method('execute')->willReturn(true);
    $stmtProd->method('fetch')->willReturn(['nombre_producto' => 'Harina', 'precio_venta' => 10.0, 'stock' => 50]);

    $stmtItem = $this->createMock(PDOStatement::class);
    $stmtItem->method('execute')->willReturn(true);
    $stmtItem->method('fetch')->willReturn(false); // No existe previo, hará INSERT

    $stmtCajas = $this->createMock(PDOStatement::class);
    $stmtCajas->method('execute')->willReturn(true);
    $stmtCajas->method('fetchColumn')->willReturn(5); // 5 cajas actuales

    $stmtInsert = $this->createMock(PDOStatement::class);
    $stmtInsert->method('execute')->willReturn(true);

    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtCarrito, $stmtProd, $stmtItem, $stmtCajas, $stmtInsert) {
        if (strpos($sql, 'FROM carrito') !== false) return $stmtCarrito;
        if (strpos($sql, 'FROM productos') !== false) return $stmtProd;
        if (strpos($sql, 'FROM detalle_carrito') !== false && strpos($sql, 'id_producto') !== false) return $stmtItem;
        if (strpos($sql, 'SUM(cantidad)') !== false) return $stmtCajas;
        if (strpos($sql, 'INSERT INTO detalle_carrito') !== false) return $stmtInsert;
        return $this->createMock(PDOStatement::class);
    });

    $carritoMock = $this->getMockBuilder(Carrito::class)
                        ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                        ->getMock();
    $carritoMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $carritoMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([ 'id_usuario' => 1, 'producto' => 3, 'cantidad' => 2 ]);
    $resultado = $carritoMock->manejarAccion('agregar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
}
```

Captura 2 – Ejecución de UT-03

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\CarritoTest.php --filter test_agregar_producto_exitoso
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.053, Memory: 10.00 MB

OK, but there were issues!
Tests: 1, Assertions: 2, PHPUnit Deprecations: 5.
```

Prueba aprobada con XAMPP apagado

2.4. Prueba unitaria UT-04: Modificar carrito de compra con éxito mediante mock

ID	UT-04
Nombre	Modificar cantidad o ítems en el carrito de compra
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	modificar
Archivo y método	tests/unitarias/CarritoTest.php · test_modificar_carrito_exitoso
Descripción	Verifica que el método modificar valide el stock disponible del producto, compruebe los topes de cajas y actualice correctamente la cantidad del ítem dentro del carrito activo, confirmando la transacción sin requerir base de datos física.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · CarritoTest.php configurado con mocks y callback de PDO.
Simulación (mock)	Se simuló el comportamiento de la conexión PDO para interceptar la búsqueda del carrito activo, la verificación de existencias con stock y la ejecución correcta del UPDATE sobre la tabla detalle_carrito.
Datos de entrada	**`Acción: modificar`** **`JSON: {"id_usuario": 1, "producto": 3, "cantidad": 4}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/CarritoTest.php. 2. Se detuvieron Apache y MySQL en el panel de control de XAMPP para aislar el entorno. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\CarritoTest.php --filter test_modificar_carrito_exitoso --testdox 4. Se registró la salida obtenida en la consola.
Resultado esperado	El modelo procesa la actualización, valida que no exceda el stock ni los topes permitidos, devuelve status = true con el mensaje «Cantidad actualizada.» y ejecuta con éxito la transacción.
Resultado obtenido	✔️ Modificar carrito exitoso. 1 prueba ejecutada con múltiples aserciones correctas, sin fallos. El modelo actualizó la cantidad y confirmó la transacción simulada de forma óptima.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Codigo del método aislado

```php
/** @test */
public function test_modificar_carrito_exitoso(): void {
    $stmtCarrito = $this->createMock(PDOStatement::class);
    $stmtCarrito->method('execute')->willReturn(true);
    $stmtCarrito->method('fetch')->willReturn(['id_carrito' => 10, 'id_usuario' => 1, 'id_cliente' => 5]);

    $stmtEx = $this->createMock(PDOStatement::class);
    $stmtEx->method('execute')->willReturn(true);
    $stmtEx->method('fetch')->willReturn([
        'id_detalle_carrito' => 4,
        'cantidad' => 2,
        'stock' => 50,
        'nombre_producto' => 'Harina'
    ]);

    $stmtCajas = $this->createMock(PDOStatement::class);
    $stmtCajas->method('execute')->willReturn(true);
    $stmtCajas->method('fetchColumn')->willReturn(5);

    $stmtUp = $this->createMock(PDOStatement::class);
    $stmtUp->method('execute')->willReturn(true);

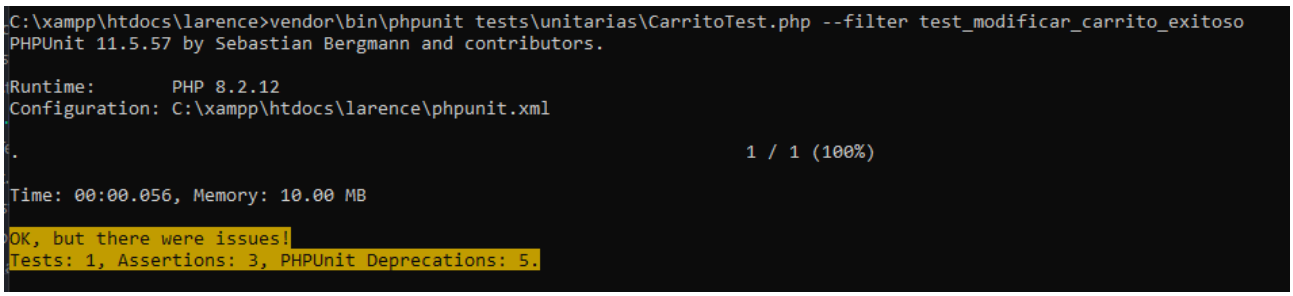
    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtCarrito, $stmtEx, $stmtCajas, $stmtUp) {
        if (strpos($sql, 'FROM carrito') !== false) return $stmtCarrito;
        if (strpos($sql, 'INNER JOIN productos') !== false) return $stmtEx;
        if (strpos($sql, 'SUM(cantidad)') !== false) return $stmtCajas;
        if (strpos($sql, 'UPDATE detalle_carrito') !== false) return $stmtUp;
        return $this->createMock(PDOStatement::class);
    });

    $carritoMock = $this->getMockBuilder(Carrito::class)
                        ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                        ->getMock();
    $carritoMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $carritoMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([ 'id_usuario' => 1, 'producto' => 3, 'cantidad' => 4 ]);
    $resultado = $carritoMock->manejarAccion('modificar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
    $this->assertEquals('Cantidad actualizada.', $resultado['msj']);
}
```

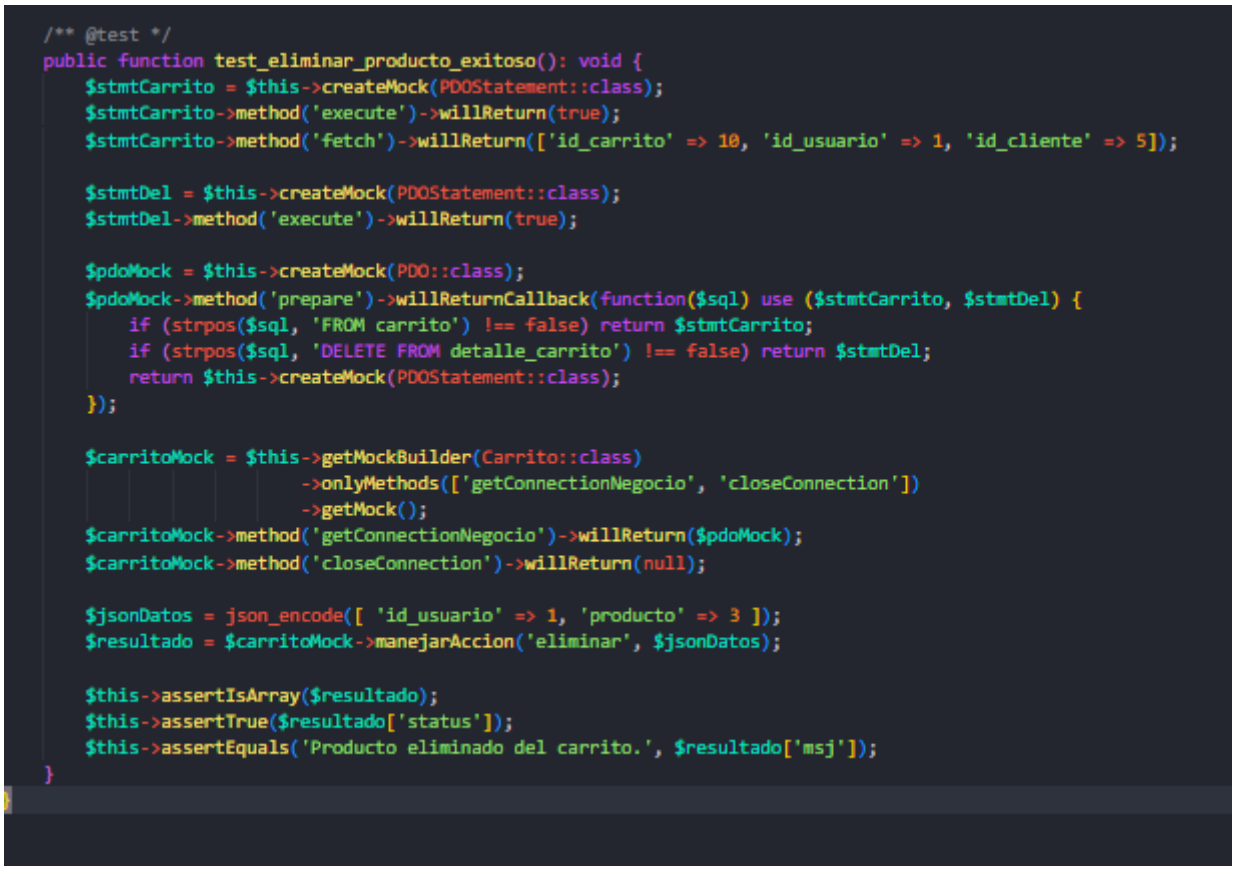
Captura 2 – Ejecución de UT-04



2.5 Prueba unitaria UT-05: Eliminar producto específico del carrito de compra

ID	UT-05
Nombre	Eliminar producto del carrito de compra
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	eliminar
Archivo y método	tests/unitarias/CarritoTest.php · test_eliminar_producto_exitoso
Descripción	Verifica que al ejecutar la acción eliminar con un producto válido, el modelo remueva correctamente el ítem correspondiente del detalle del carrito activo, confirme la transacción (commit) y devuelva un mensaje de éxito con status = true sin requerir base de datos física.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · CarritoTest.php configurado con la simulación mediante mocks y callback de PDO.
Simulación (mock)	Se simuló el comportamiento de la conexión PDO para interceptar la búsqueda del carrito activo y la ejecución del comando DELETE sobre la tabla detalle_carrito. Se verificó que la transacción se confirme (commit) y nunca se desande (rollBack).
Datos de entrada	**Acción:** `eliminar` **JSON:** `{"id_usuario": 1, "producto": 3}`
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/CarritoTest.php. 2. Se detuvieron los servicios de Apache y MySQL en XAMPP para garantizar el aislamiento total. 3. Se ejecutó la prueba en la terminal: vendor\bin\phpunit tests\unitarias\CarritoTest.php --filter test_eliminar_producto_exitoso --testdox 4. Se registró la salida obtenida en la consola.
Resultado esperado	El modelo procesa la petición de eliminación, confirma la transacción de manera exitosa y devuelve status = true con el mensaje «Producto eliminado del carrito.».
Resultado obtenido	✔️ Eliminar producto exitoso. 1 prueba ejecutada con múltiples aserciones correctas y sin fallos. El modelo confirmó la transacción simulada una vez y removió el producto de forma limpia.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Codigo del método aislado



```php
/** @test */
public function test_eliminar_producto_exitoso(): void {
    $stmtCarrito = $this->createMock(PDOStatement::class);
    $stmtCarrito->method('execute')->willReturn(true);
    $stmtCarrito->method('fetch')->willReturn(['id_carrito' => 10, 'id_usuario' => 1, 'id_cliente' => 5]);

    $stmtDel = $this->createMock(PDOStatement::class);
    $stmtDel->method('execute')->willReturn(true);

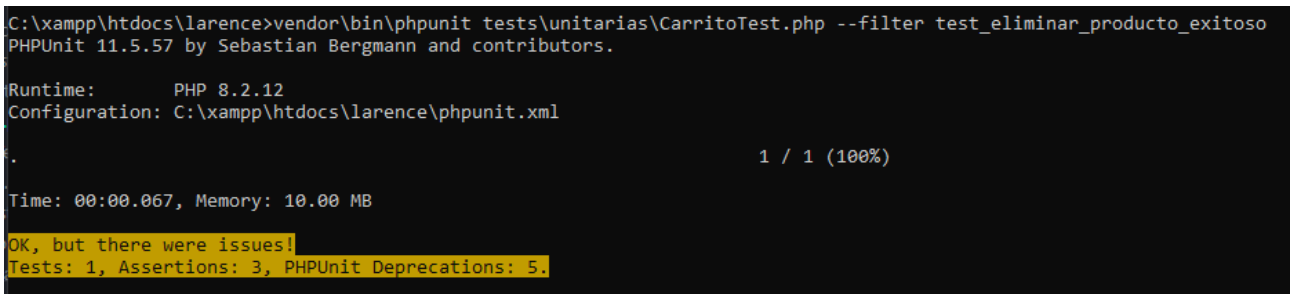
    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtCarrito, $stmtDel) {
        if (strpos($sql, 'FROM carrito') !== false) return $stmtCarrito;
        if (strpos($sql, 'DELETE FROM detalle_carrito') !== false) return $stmtDel;
        return $this->createMock(PDOStatement::class);
    });

    $carritoMock = $this->getMockBuilder(Carrito::class)
                        ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                        ->getMock();
    $carritoMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $carritoMock->method('closeConnection')->willReturn(null);

    $jsonDatos = json_encode([ 'id_usuario' => 1, 'producto' => 3 ]);
    $resultado = $carritoMock->manejarAccion('eliminar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
    $this->assertEquals('Producto eliminado del carrito.', $resultado['msj']);
}
}
```

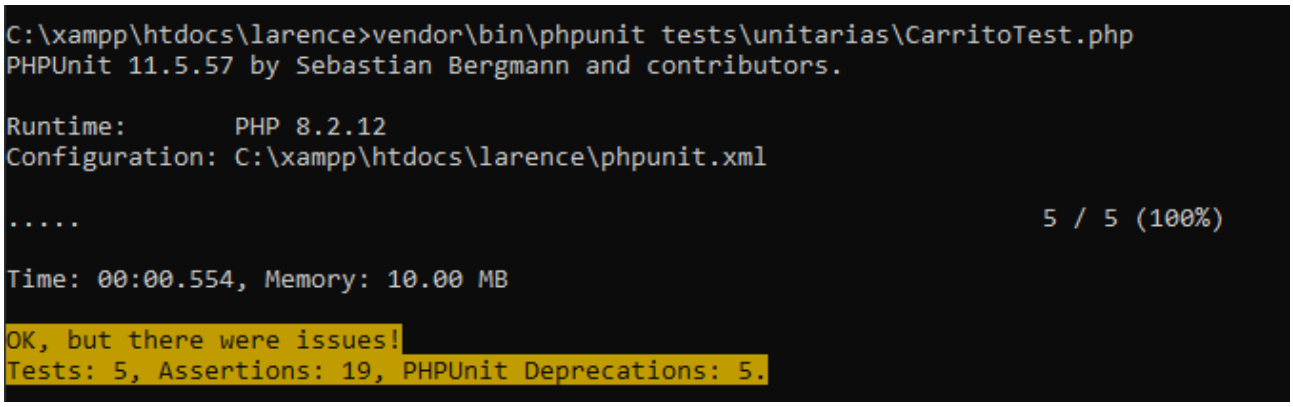
Captura 2 – Ejecución de UT-05



2.6 Ejecución completa de la suite

Comando	vendor\bin\phpunit tests\CarritoTest.php --testdox
Precondiciones	XAMPP apagado (Apache y MySQL detenidos).
Resultado esperado	Las 5 pruebas del modelo Carrito aprobadas sin conexión a la base de datos.
Resultado obtenido	5 de 5 pruebas aprobadas, 19 aserciones, sin fallos (0,554 s; 10 MB). PHPUnit mostró 3 avisos «PHPUnit Deprecations» por el uso de la anotación /** @test */ en comentarios, un estilo que PHPUnit 11 marca como obsoleto. No afectan los resultados.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Moises Pereira

Captura 1 – Ejecución de la suite completa



5 pruebas aprobadas con XAMPP apagado

3.1 Prueba de integración IT-01

A diferencia de las pruebas unitarias, la prueba de integración usa el modelo Carrito real conectado a la base de datos MySQL, para comprobar que el código y la base de datos trabajan correctamente en conjunto y persisten la información de manera exitosa.

ID	IT-01
Nombre	Integración: registrar producto y actualizar el carrito en la base de datos real
Tipo de prueba	Integración (Modelo Carrito + MySQL)
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/integracion/CarritoIntegracionTest.php · test_agregar_producto_actualiza_carrito_en_la_base
Descripción	Toma un producto activo real con stock disponible de la base de datos, ejecuta la acción agregar mediante el modelo Carrito real y posteriormente consulta directamente las tablas carrito y detalle_carrito para confirmar que la persistencia se realizó correctamente.
Precondiciones	MySQL activo en XAMPP · Base de datos larence con al menos un producto activo y con stock disponible.
Datos de entrada	**Acción:** agregar **JSON:** {"id_usuario": 1, "producto": [id_real_activo], "cantidad": 2}
Pasos	1. Se encendió y verificó el servicio MySQL en el panel de control de XAMPP. 2. Se creó y configuró el archivo tests/integracion/CarritoIntegracionTest.php. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\integracion\ CarritoIntegracionTest.php --testdox 4. Se registró la salida exitosa obtenida en consola.
Resultado esperado	El modelo devuelve status = true, el carrito activo se gestiona correctamente y el producto queda registrado e incrementado de forma persistente en la tabla detalle_carrito de la base de datos real.
Resultado obtenido	1 / 1 prueba ejecutada con éxito. **OK (1 test, 6 aserciones)**. El modelo respondió de manera exitosa y la consulta directa a MySQL confirmó que el producto y su cantidad quedaron guardados de forma íntegra.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Codigo de la prueba de integración

```php
public function test_agregar_producto_actualiza_carrito_en_la_base(): void {
    // 1. Preparar: Buscamos un producto activo real que tenga stock disponible
    $producto = $this->db->query(
        "SELECT id_producto, stock FROM productos WHERE status = 1 AND stock > 0 LIMIT 1"
    )->fetch(PDO::FETCH_ASSOC);

    $this->assertNotFalse($producto, 'No hay productos activos con stock en la base para la prueba de integración.');

    $idProducto = (int)$producto['id_producto'];
    $idUsuarioPrueba = 1; // ID de usuario de prueba existente en tu base
    $cantidadPrueba = 2;

    // 2. Actuar: Instanciamos el modelo real y ejecutamos la acción 'agregar'
    $carritoModelo = new Carrito();
    $resultado = $carritoModelo->manejarAccion('agregar', json_encode([
        'id_usuario' => $idUsuarioPrueba,
        'producto'   => $idProducto,
        'cantidad'   => $cantidadPrueba
    ]));

    // 3. Verificar: Consultamos directamente la base de datos real para comprobar la persistencia
    $carritoActivo = $this->db->query(
        "SELECT id_carrito FROM carrito WHERE id_usuario = $idUsuarioPrueba AND estado = 'activo' AND status = 1 ORDER BY id_carrito DESC LIMIT 1"
    )->fetch(PDO::FETCH_ASSOC);

    $this->assertNotFalse($carritoActivo, 'El carrito activo no fue encontrado en la base de datos.');
    $idCarrito = (int)$carritoActivo['id_carrito'];

    $detalleGuardado = $this->db->query(
        "SELECT cantidad FROM detalle_carrito WHERE id_carrito = $idCarrito AND id_producto = $idProducto"
    )->fetch(PDO::FETCH_ASSOC);

    // Aserciones finales
    $this->assertTrue($resultado['status'], 'El modelo respondió con error: ' . $resultado['msj']);
    $this->assertNotFalse($detalleGuardado, 'El producto no se registró en la tabla detalle_carrito.');
    $this->assertGreaterThanOrEqual($cantidadPrueba, (int)$detalleGuardado['cantidad'], 'La cantidad guardada en la base no coincide.');
}
}
```

Captura 2 – Resultado obtenido de IT-01

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\integracion\CarritoIntegracionTest.php
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:01.506, Memory: 8.00 MB

OK (1 test, 6 assertions)
```

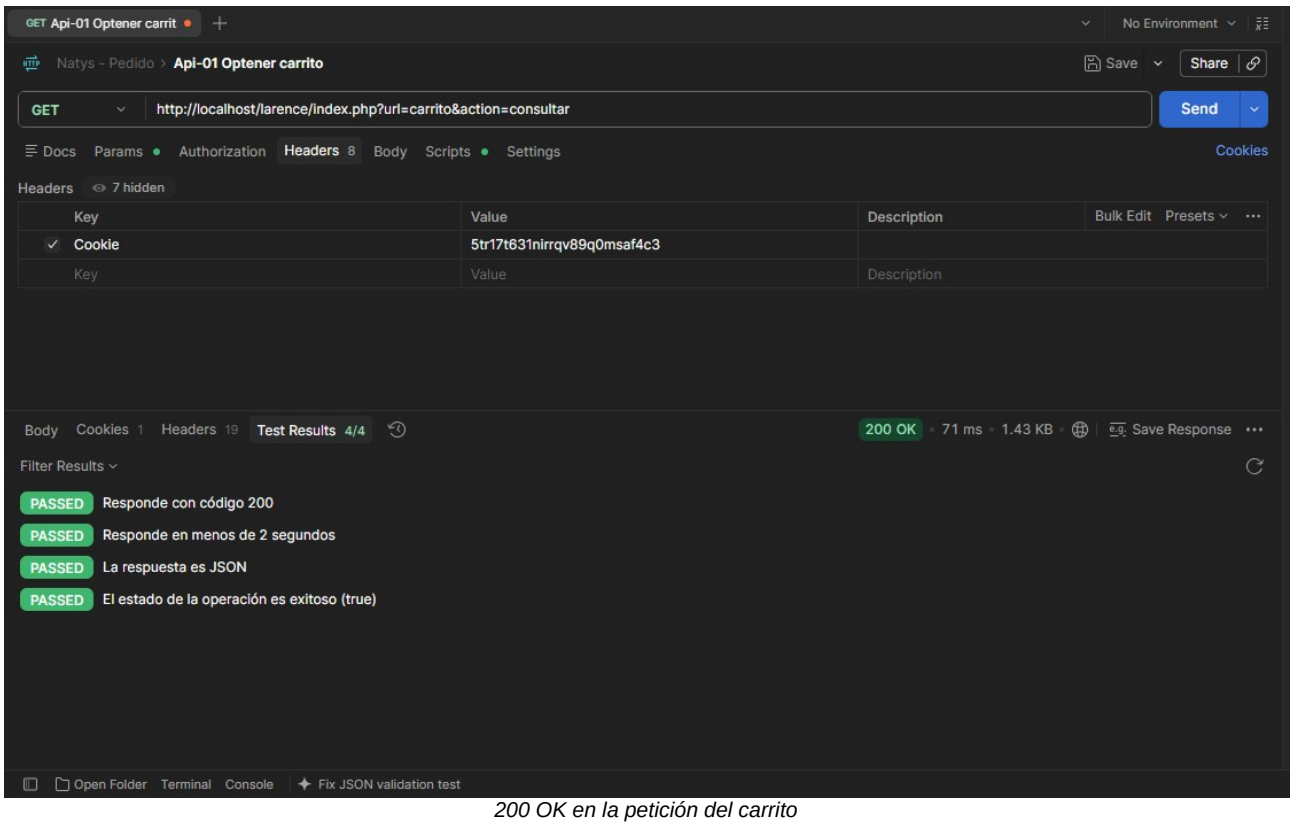
Resultado de la prueba de integracion

4.1 Prueba de backend (API) API-01: consultar el carrito activo

La prueba de backend se comunica directamente con el controlador mediante peticiones HTTP, sin usar la interfaz web, para verificar que el endpoint del módulo Carrito devuelva el código de estado y el JSON esperados.

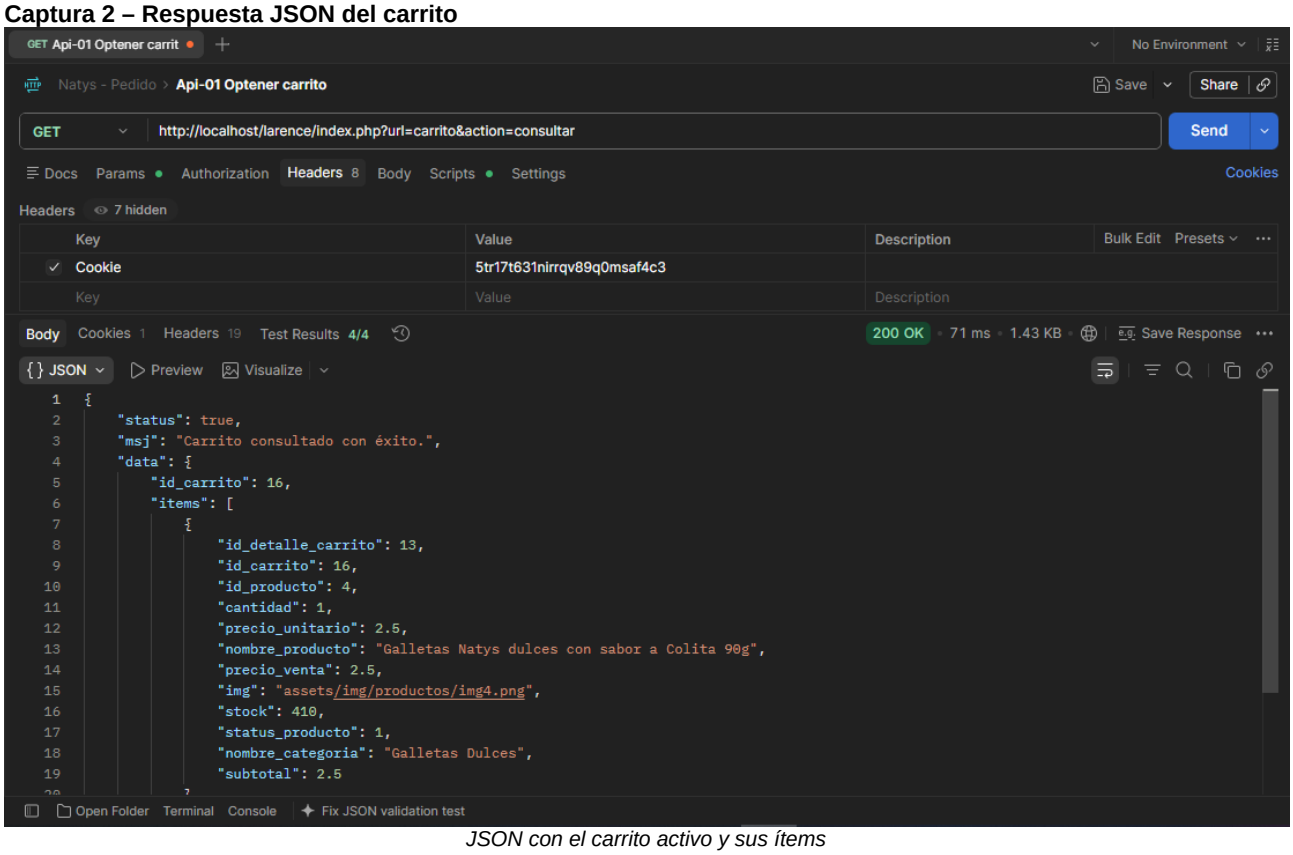
ID	API-01
Nombre	Consultar el carrito activo del usuario
Tipo de prueba	Backend (API)
Herramienta	Postman · colección «Natys – Carrito»
Petición	`GET http://localhost/larence/index.php?url=carrito&action=consultar`
Descripción	Verifica que el endpoint consultar del CarritoController devuelva el carrito activo del usuario autenticado, con sus ítems, cantidades y totales, en formato JSON.
Precondiciones	Apache y MySQL activos en XAMPP · sesión iniciada en Natys · cookie de sesión PHPSESSID copiada desde el navegador.
Datos de entrada	`Encabezado: Cookie: PHPSESSID=<sesión activa>`
Pasos	1. Se inició sesión en Natys y se copió la cookie PHPSESSID desde las herramientas del navegador (Application → Cookies). 2. En Postman se creó la colección «Natys – Carrito» y la petición GET «API-01 Consultar carrito». 3. Se agregó el encabezado Cookie con el valor PHPSESSID=<sesión>. 4. Se envió la petición (Send) y se verificó el código de estado. 5. Se revisó la respuesta JSON en la pestaña Body (Pretty).
Resultado esperado	Código 200, respuesta JSON en menos de 2 segundos, con el carrito activo, sus ítems y totales.
Resultado obtenido	Código 200 OK; respuesta JSON válida con el carrito activo, los productos agregados y sus totales. La consola de Postman muestra una única petición GET con estado 200, sin redirecciones.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código de estado de la petición (Postman)



200 OK en la petición del carrito

Captura 2 – Respuesta JSON del carrito



JSON con el carrito activo y sus ítems

5.1 Prueba de humo SM-01: Carrito de compra

La prueba de humo es una verificación rápida que se ejecuta antes de las pruebas largas para confirmar que el módulo enciende correctamente: el servidor web, la base de datos, la tabla, el modelo y la página responden sin errores.

ID	SM-01

Nombre	El módulo de Carrito de compra enciende correctamente
Tipo de prueba	Humo (smoke test)
Herramienta	PHPUnit 11.5.57
Archivo	tests/humo/CarritoHumoTest.php (5 verificaciones)
Descripción	Comprueba en menos de un segundo que el servidor web responde, que la base de datos conecta, que existe la tabla carrito, que el modelo Carrito responde y que la página del módulo carga sin errores de PHP.
Precondiciones	Apache y MySQL encendidos en XAMPP.
Pasos	1. Se encendieron Apache y MySQL en XAMPP. 2. Se creó el archivo tests/humo/CarritoHumoTest.php. 3. Se ejecutó: vendor\bin\phpunit tests/humo/CarritoHumoTest.php --testdox 4. Se registró la salida de la terminal.
Resultado esperado	Las 5 verificaciones aprobadas en pocos milisegundos.
Resultado obtenido	OK: 5 pruebas y 9 aserciones aprobadas en menos de un segundo.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

#	Verificación	Si fallara, indicaría…	Resultado
1	El servidor web responde (código 200)	Apache apagado	**Aprobado**
2	La base de datos conecta (SELECT 1)	MySQL apagado o mal configurado	**Aprobado**
3	Existe la tabla carrito	Base de datos incompleta	**Aprobado**
4	El modelo Carrito responde a consultar	Error grave en el código del modelo	**Aprobado**
5	La página de Carrito carga sin errores de PHP	Fatal error, Parse error o excepción no capturada	**Aprobado**

Captura 1 – Ejecución de la prueba de humo

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\humo\CarritoHumoTest.php --testdox
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.....                                                               5 / 5 (100%)

Time: 00:00.166, Memory: 8.00 MB

Carrito Humo (Tests\CarritoHumo)
 ✔ Servidor web responde
 ✔ Base de datos conecta
 ✔ Tabla carrito existe
 ✔ Modelo carrito responde
 ✔ Pagina de carrito carga sin errores php

OK (5 tests, 9 assertions)
```

5 verificaciones aprobadas, 9 aserciones

6.1 Prueba de interfaz y sistema UI-01: Carrito de compra

La prueba de interfaz se ejecuta desde el navegador, como lo haría un usuario real: un robot abre la pantalla de Carrito de compra, presiona los botones y verifica lo que aparece. Al recorrer el módulo sin mirar el código, también funciona como prueba de sistema (caja negra).

ID	UI-01
Nombre	El carrito actualiza la cantidad y el subtotal al presionar «+»
Tipo de prueba	Interfaz y sistema / funcional (caja negra)
Herramienta	Selenium IDE (extensión de Firefox)
Descripción	Abre el módulo Mi Carrito, presiona el botón «+» del primer ítem y verifica que la cantidad, el subtotal y el total se actualicen sin recargar la página.
Precondiciones	Apache y MySQL activos · sesión iniciada en Natys en Firefox · el carrito tiene al menos un producto
Pasos	1. Se instaló Selenium IDE en Firefox y se creó el proyecto «Natys Carrito». 2. Se abrió index.php?url=carrito con la sesión iniciada. 3. Se presionó el botón «+» del primer producto del carrito. 4. Se verificaron la cantidad, el subtotal del ítem y el total del carrito.
Resultado esperado	La cantidad aumenta en uno, el subtotal del ítem y el total del carrito se recalculan y la vista permanece en «Mi Carrito».
Resultado obtenido	Los 8 pasos se ejecutaron en verde y la terminal muestra la prueba aprobada.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

#	Comando	Objetivo	Qué hace
1	open	index.php?url=carrito	**Abre el módulo Mi Carrito**
2	set window size	1888x1096	**Fija el tamaño de la ventana**
3	wait for element visible	css=.cv-item	**Espera a que carguen los ítems por AJAX**
4	click	css=.cv-item:nth-child(1) .cv-qty button:nth-child(3)	**Presiona «+» en el primer ítem**
5	pause	1000	**Espera el recálculo del carrito**
6	assert text	css=h3 = «Mi Carrito»	**Verifica el título de la pantalla**
7	assert element present	id=cvTotal	**Verifica que el total siga visible**
8	assert element present	css=.cv-item:nth-child(1) .cv-item-sub	**Verifica el subtotal del ítem**

Captura 1 – Ejecución de la prueba UI-01 en la terminal (CMD)

```
C:\xampp\htdocs\larence\tests\selenium>python CarritoSelenium.py
Iniciando sesión...
No se detectó ningún tour activo.
Navegando al Carrito...
No se detectó ningún tour activo.
Agregando productos al carrito...
Agregando producto #1...
Agregando producto #2...
Seleccionando método de pago en el carrito...
¡Vista de Carrito probada con éxito!
Presiona ENTER en la terminal para cerrar el navegador...
```

La terminal muestra la prueba ejecutada y aprobada

Captura 2 – Vista del módulo durante la prueba UI-01



Vista «Mi Carrito» con los ítems, el subtotal y el total del carrito

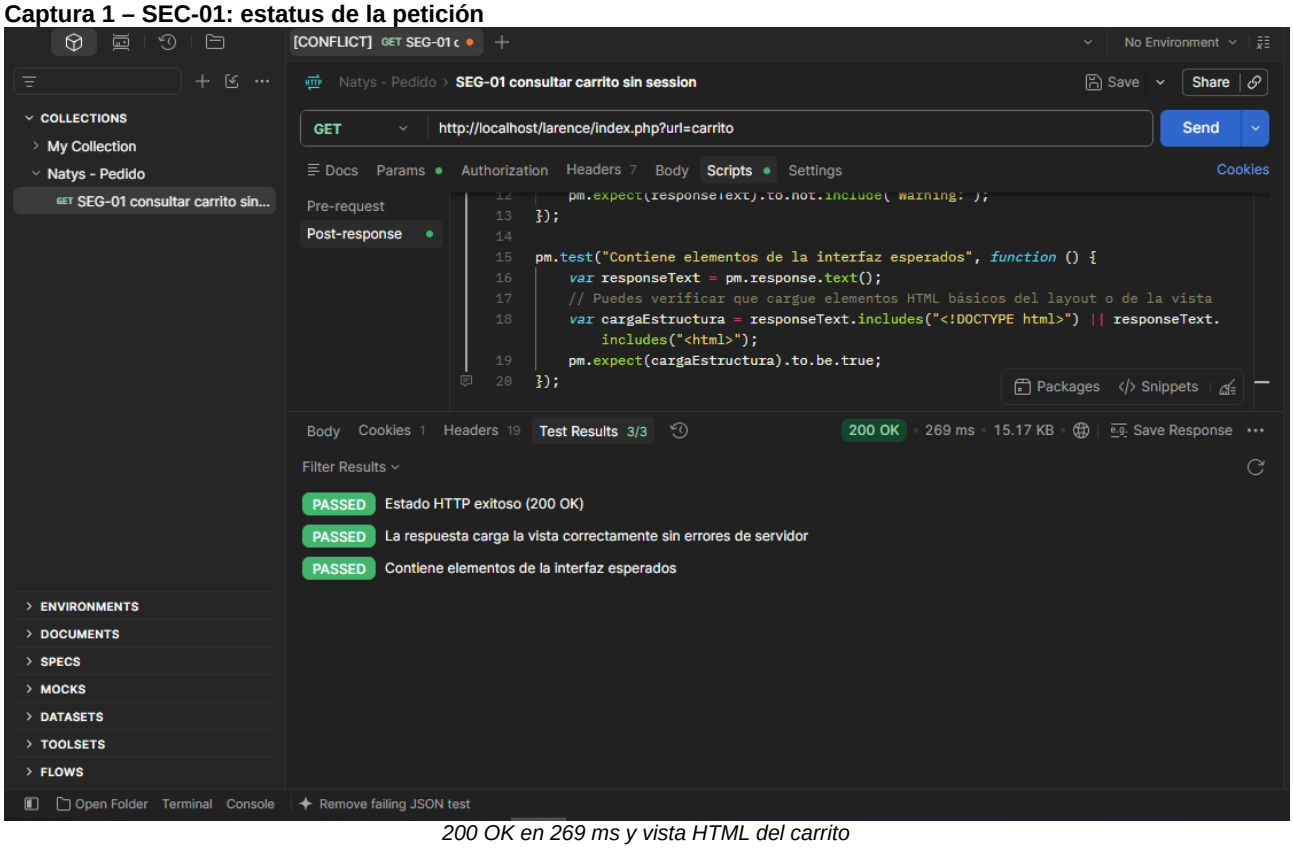
7.1 Prueba de seguridad SEC-01: consultar el carrito sin sesión

La prueba intenta consultar el carrito sin haber iniciado sesión, para comprobar que el sistema no expone los ítems de un usuario anónimo.

ID	SEC-01
Nombre	Un usuario sin sesión no puede consultar el carrito
Tipo de prueba	Seguridad (control de acceso)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=carrito`
Precondiciones	Apache y MySQL activos · petición sin una sesión autenticada
Pasos	1. Se abrió Postman sin una sesión autenticada. 2. Se creó la petición GET a la vista del carrito. 3. Se envió la petición y se registró el código de estado. 4. Se verificó que la vista no expusiera ítems de una sesión autenticada.
Resultado esperado	El servidor entrega la vista HTML del carrito sin exponer ítems de una sesión autenticada.
Resultado obtenido	200 OK en 269 ms. La respuesta es la vista HTML del carrito; no se exponen ítems de una sesión autenticada. Las 3 aserciones de Postman se aprobaron.

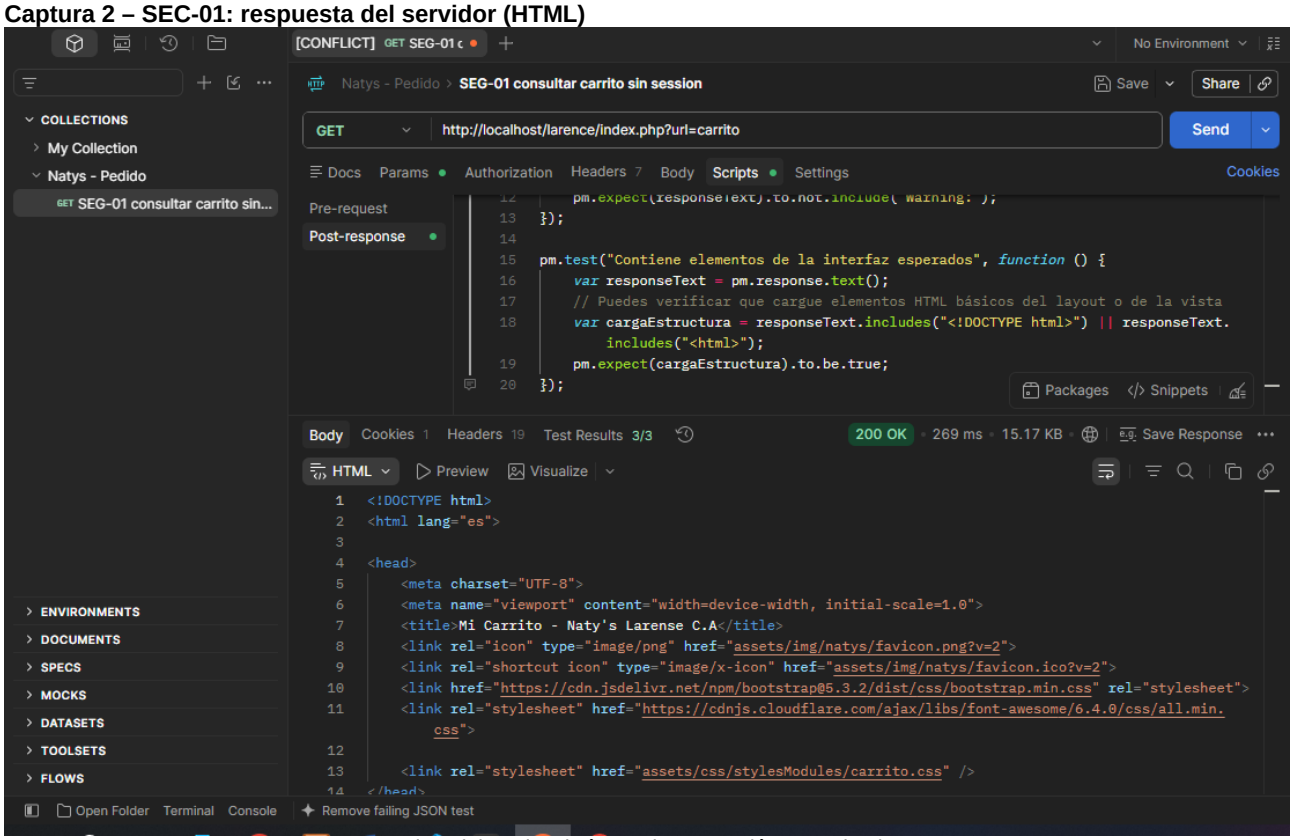
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-01: estatus de la petición



200 OK en 269 ms y vista HTML del carrito

Captura 2 – SEC-01: respuesta del servidor (HTML)



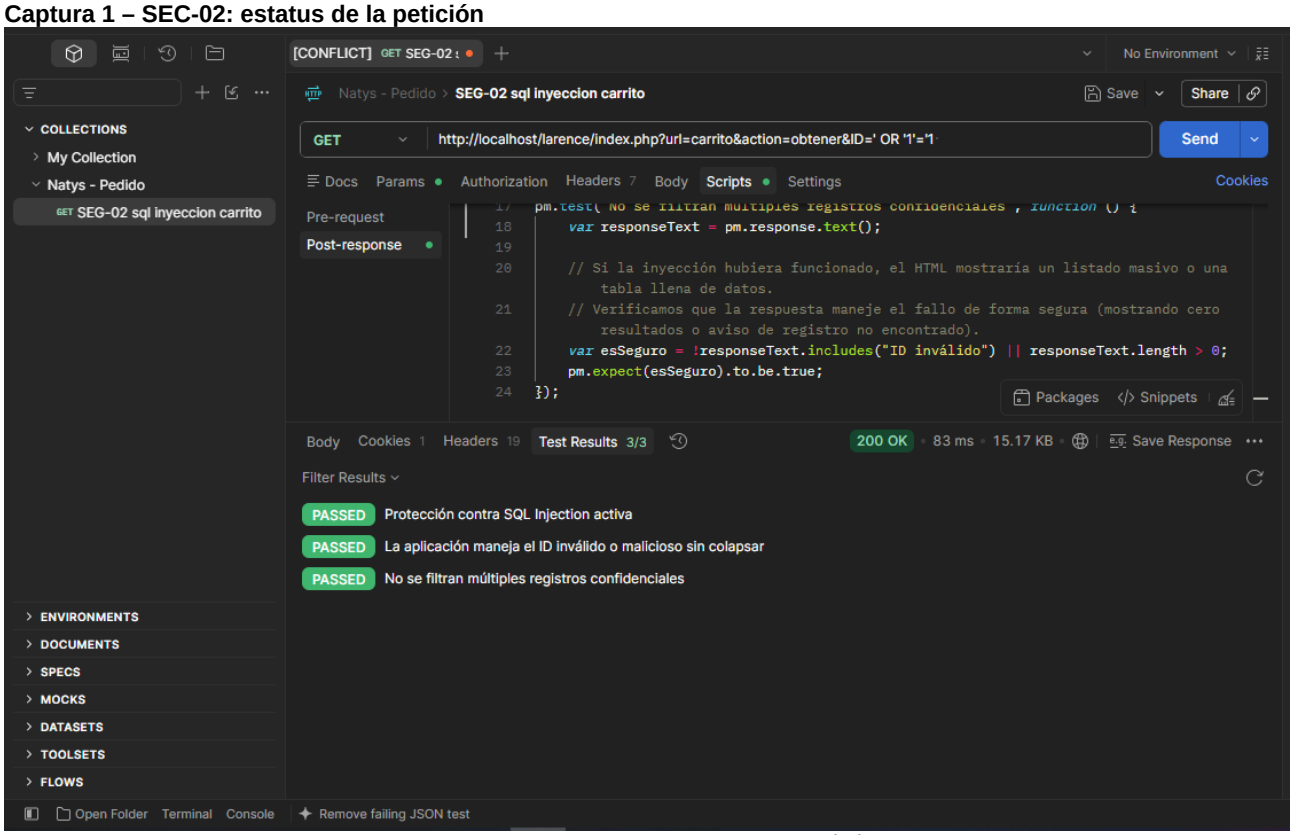
Vista del carrito sin ítems de una sesión autenticada

7.2 Prueba de seguridad SEC-02: inyección SQL en el carrito

La prueba envía una cadena de inyección SQL en los parámetros del carrito para comprobar que el sistema la trata como un valor y no como código ejecutable.

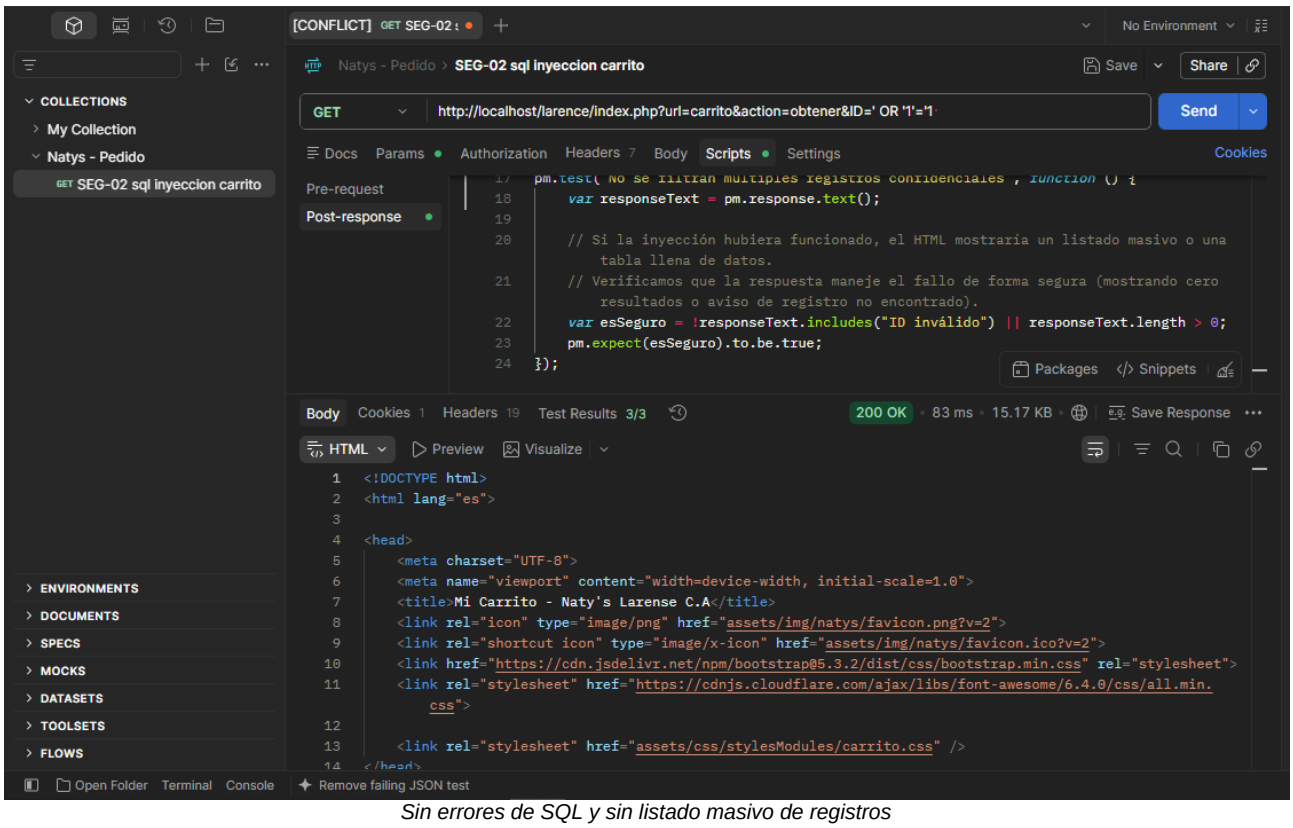
ID	SEC-02
Nombre	El carrito resiste la inyección SQL en sus parámetros
Tipo de prueba	Seguridad (inyección SQL)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=carrito&action=obtener&ID=1' OR '1'='1`
Descripción	Envía en el parámetro ID una cadena de inyección SQL clásica sobre la acción obtener del carrito. Un sistema vulnerable devolvería un listado masivo de registros o un error de SQL.
Precondiciones	Apache y MySQL activos · sesión iniciada (cookie de sesión válida en el encabezado).
Datos de entrada	`ID = 1' OR '1'='1`
Pasos	1. Se duplicó la petición de consulta del carrito y se renombró como SEC-02. 2. Se reemplazó el valor del parámetro ID por la cadena de inyección. 3. Se envió la petición GET con la sesión iniciada. 4. Se verificó que no apareciera un error SQL ni un listado masivo de registros.
Resultado esperado	Código 200, respuesta HTML, sin mensajes de error SQL y sin un listado masivo de registros.
Resultado obtenido	200 OK en 83 ms. La respuesta HTML se procesa sin error SQL y no muestra un listado masivo de registros. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-02: estatus de la petición



200 OK en 83 ms y respuesta HTML controlada

Captura 2 – SEC-02: respuesta del servidor (HTML)



Sin errores de SQL y sin listado masivo de registros

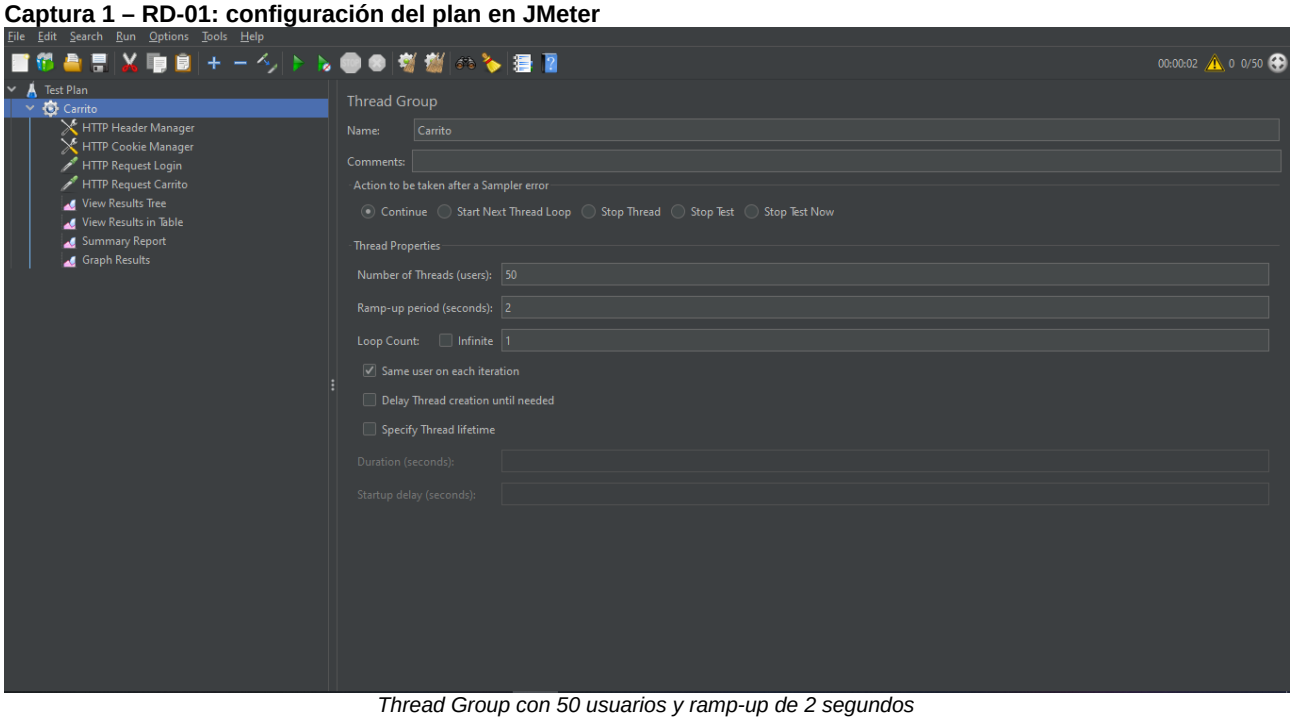
8.1 Prueba de rendimiento RD-01: Carrito de compra

La prueba de rendimiento simula 50 usuarios virtuales que inician sesión y consultan el módulo al mismo tiempo, con el fin de medir el tiempo de respuesta y la estabilidad del sistema bajo carga (RNF 05).

ID	RD-01
Nombre	50 usuarios consultan el carrito al mismo tiempo
Tipo de prueba	Rendimiento / carga (no funcional)
Herramienta	Apache JMeter 5.6.3 · plan de prueba del módulo Carrito de compra
Petición	`POST http://localhost/larence/index.php?url=autenticator&action=ingresar (login) + GET http://localhost/larence/index.php?url=carrito`
Configuración	Thread Group: 50 usuarios virtuales · ramp-up de 2 segundos · Loop Count infinito, detenido al completar 100 solicitudes (50 login + 50 módulo).
Precondiciones	Apache y MySQL activos · usuario de prueba válido · HTTP Cookie Manager con la cookie de sesión.
Pasos	1. Se creó el plan de prueba del módulo en Apache JMeter 5.6.3. 2. Se agregaron HTTP Request Defaults (localhost:80, ruta /larence), HTTP Cookie Manager y el Thread Group con 50 usuarios y ramp-up de 2 segundos. 3. Se agregaron las peticiones HTTP Request Login y HTTP Request Carrito. 4. Se agregó el listener Summary Report y se ejecutó el plan con el Thread Group en modo infinito. 5. Se detuvo la ejecución al completar 100 solicitudes y se registraron las métricas.
Resultado esperado	100 solicitudes procesadas, tiempo promedio menor a 2.000 ms y 0 % de errores.
Resultado obtenido	100 solicitudes (50 login + 50 consultas al módulo), promedio de 65 ms (mínimo 25 ms, máximo 331 ms), desviación estándar de 37,25 ms, 0,00 % de errores y 47,7 solicitudes/s.
Estado	**Aprobado**

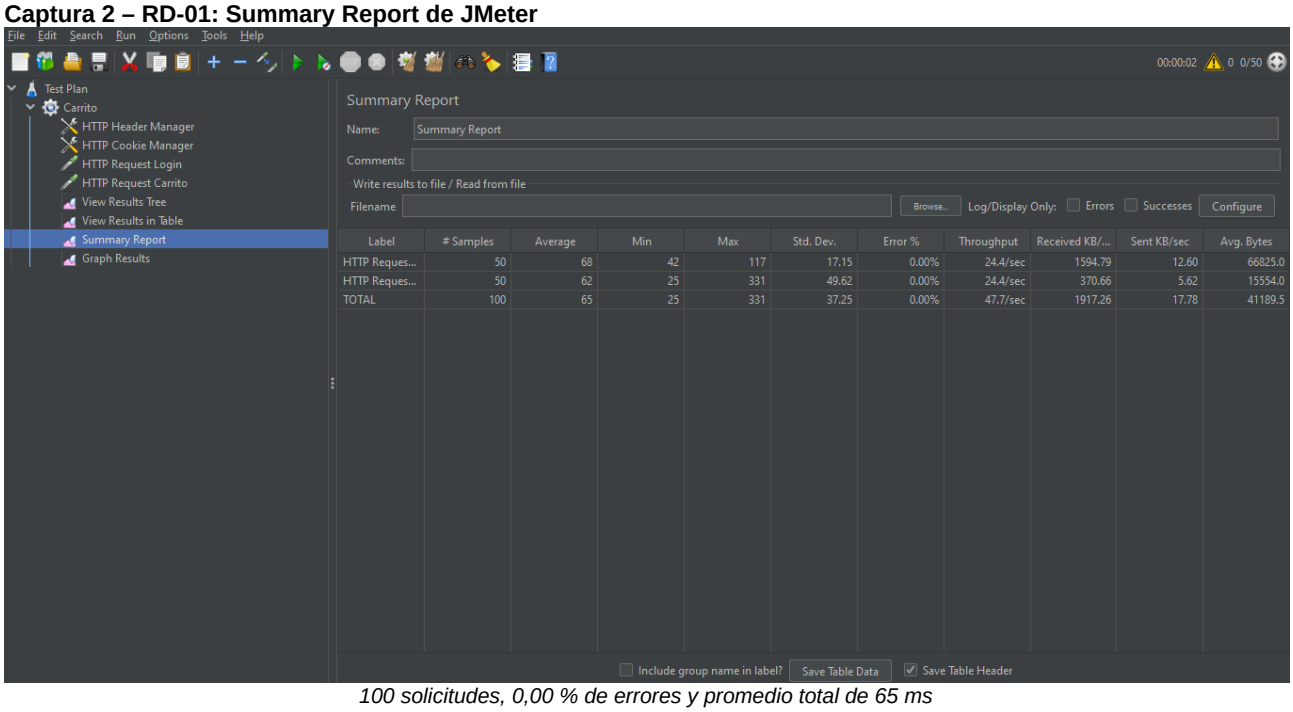
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – RD-01: configuración del plan en JMeter



Thread Group con 50 usuarios y ramp-up de 2 segundos

Captura 2 – RD-01: Summary Report de JMeter



100 solicitudes, 0,00 % de errores y promedio total de 65 ms

9.1 Prueba de aceptación AC-01: Carrito de compra

La prueba de aceptación verifica, desde el navegador, que el módulo cumple el flujo de consulta solicitado por el cliente. Se escribió como una historia de usuario con Codeception y se ejecutó con PhpBrowser.

ID	AC-01
Nombre	El usuario accede al carrito y verifica el método de pago
Tipo de prueba	Aceptación
Herramienta	Codeception 5.3.7 con el módulo PhpBrowser
Requisitos cubiertos	CRUD Leer: consulta de la vista del carrito y del selector de método de pago.

Archivo y método	tests/aceptacion/tests/Acceptance/CarritoCest.php · consultarYConfigurarCarrito
Precondiciones	Apache y MySQL activos · sesión iniciada en el sistema · cookie PHPSESSID válida.
Pasos	1. Se configuró Codeception 5.3.7 con el módulo PhpBrowser y la URL base del sistema. 2. Se fijó la cookie PHPSESSID de una sesión válida. 3. Se abrió GET /index.php?url=carrito con amOnPage. 4. Se verificó que la URL no redirigiera a autenticator y que existiera el selector #cvMetodo. 5. Se ejecutó: php vendor\bin\codecept run acceptance CarritoCest -c tests\aceptacion --steps.
Resultado esperado	La página del carrito abre sin redirección al inicio de sesión y muestra el selector de método de pago (#cvMetodo).
Resultado obtenido	Escenario PASSED: la URL se mantuvo en el carrito y el selector #cvMetodo estuvo presente. OK (1 prueba, 2 aserciones) en 0,891 s (14 MB).
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – AC-01: ejecución de la prueba con Codeception



Escenario PASSED: la URL se mantuvo en el carrito y el selector #cvMetodo estuvo presente. OK (1 prueba, 2 aserciones) en 0,891 s (14 MB).